

1.Single Inheritance – Scenario

```
class Vehicle {
    void startEngine() {
        System.out.println("Engine Started");
    }
    void stopEngine() {
        System.out.println("Engine Stopped");
    }
}
class Car extends Vehicle {
    @Override
    void startEngine() {
        super.startEngine();
        System.out.println("Engine start time logged");
    }
    void playMusic() {
        System.out.println("Playing Music");
    }
}
public class SingleInheritance {
    public static void main(String[] args) {
        Car c = new Car();
        c.startEngine();
        c.playMusic();
        c.stopEngine();
    }
}
```

OUTPUT:

```
Engine Started
Engine start time logged
Playing Music
Engine Stopped
```

2. Multilevel Inheritance – Scenario

```
class Employee {
    int empId = 101;
    String name = "Prudhvi";
    void displayEmployee() {
        System.out.println("Employee ID : " + empId);
        System.out.println("Employee Name : " + name);
    }
}
class Developer extends Employee {
```

```

String language = "Java";
void displayLanguage() {
    System.out.println("Programming Language : " + language);
}
}
class SeniorDeveloper extends Developer {
    int teamSize = 5;
    void codeReview() {
        System.out.println("Performing Code Review");
        System.out.println("Team Size : " + teamSize);
    }
}
public class MultilevelInheritance {
    public static void main(String[] args) {
        SeniorDeveloper sd = new SeniorDeveloper();
        sd.displayEmployee();
        sd.displayLanguage();
        sd.codeReview();
    }
}

```

OUTPUT:

```

Employee ID : 101
Employee Name : Prudhvi
Programming Language : Java
Performing Code Review
Team Size : 5

```

3. Hierarchical Inheritance – Scenario

```

class Payment {
    void validate() {
        System.out.println("Payment Validation Successful");
    }
}
class CreditCardPayment extends Payment {
    void processPayment() {
        System.out.println("Credit Card Payment Processed");
    }
}
class UPIPayment extends Payment {
    void processPayment() {
        System.out.println("UPI Payment Processed");
    }
}
class NetBankingPayment extends Payment {

```

```

        void processPayment() {
            System.out.println("Net Banking Payment Processed");
        }
    }
}
public class HierarchicalInheritance {
    public static void main(String[] args) {
        CreditCardPayment c = new CreditCardPayment();
        c.validate();
        c.processPayment();
        System.out.println();
        UPIPayment u = new UPIPayment();
        u.validate();
        u.processPayment();
        System.out.println();
        NetBankingPayment n = new NetBankingPayment();
        n.validate();
        n.processPayment();
    }
}

```

OUTPUT:

Payment Validation Successful
UPI Payment Processed

Payment Validation Successful
Net Banking Payment Processed

4. Compile-Time Polymorphism (Method Overloading) –Scenario

```

class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
    double add(double a, double b) {
        return a + b;
    }
}
public class MethodOverloading {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("Two Integers: " + c.add(5, 10));
        System.out.println("Three Integers: " + c.add(5, 10, 15));
        System.out.println("Two Decimals: " + c.add(5.5, 10.5));
    }
}

```

```

        System.out.println("Mixed Values: " + c.add(5, 10.0));
    }
}

```

OUTPUT:

Two Integers: 15
 Three Integers: 30
 Two Decimals: 16.0
 Mixed Values: 15.0

5. Runtime Polymorphism (Method Overriding) –Scenario

```

class Restaurant {
    void calculateDeliveryTime() {
        System.out.println("Calculating Delivery Time");
    }
}
class Pizza extends Restaurant {
    @Override
    void calculateDeliveryTime() {
        System.out.println("Pizza Delivery Time: 30 Minutes");
    }
}
class Burger extends Restaurant {
    @Override
    void calculateDeliveryTime() {
        System.out.println("Burger Delivery Time: 20 Minutes");
    }
}
class Biryani extends Restaurant {
    @Override
    void calculateDeliveryTime() {
        System.out.println("Biryani Delivery Time: 40 Minutes");
    }
}
public class MethodOverriding {
    public static void main(String[] args) {
        Restaurant r;
        r = new Pizza();
        r.calculateDeliveryTime();
        r = new Burger();
        r.calculateDeliveryTime();
        r = new Biryani();
        r.calculateDeliveryTime();
    }
}

```

OUTPUT:

Pizza Delivery Time: 30 Minutes

Burger Delivery Time: 20 Minutes

Biryani Delivery Time: 40 Minutes

6. Interface – Scenario

```
interface SmartDevice {
    void turnOn();
    void turnOff();
}
class SmartBulb implements SmartDevice {
    public void turnOn() {
        System.out.println("Smart Bulb ON");
    }
    public void turnOff() {
        System.out.println("Smart Bulb OFF");
    }
}
class SmartFan implements SmartDevice {
    public void turnOn() {
        System.out.println("Smart Fan ON");
    }
    public void turnOff() {
        System.out.println("Smart Fan OFF");
    }
}
class SmartAC implements SmartDevice {
    public void turnOn() {
        System.out.println("Smart AC ON");
    }
    public void turnOff() {
        System.out.println("Smart AC OFF");
    }
}
public class InterfaceExample {
    public static void main(String[] args) {
        SmartDevice bulb = new SmartBulb();
        bulb.turnOn();
        bulb.turnOff();
        System.out.println();
        SmartDevice fan = new SmartFan();
        fan.turnOn();
        fan.turnOff();
        System.out.println();
    }
}
```

```
        SmartDevice ac = new SmartAC();  
        ac.turnOn();  
        ac.turnOff();  
    }  
}
```

OUTPUT:

Smart Fan ON
Smart Fan OFF

Smart AC ON
Smart AC OFF